

PRD: Adobe Competitive Intelligence Automation

Status: Active — Phase 1 Complete

Version: 2.0

Last Updated: April 2026

Repo: github.com/bnamatherdhala7/Competitive-research-Report-using-Claude

1. SUMMARY

An end-to-end system that generates branded, acquisition-focused competitive intelligence PDFs for Adobe product teams (Acrobat, Express, Firefly). Phase 1 is live: three full playbooks covering pricing, messaging, battlecards, and PLG motions are generated by Claude using live web research and exported as Adobe-branded PDFs in under 60 seconds. Phase 2 delivers reports automatically to Slack channels on a monthly schedule. Phase 3 builds a full intelligence platform with trend tracking, dashboard, and cross-team distribution.

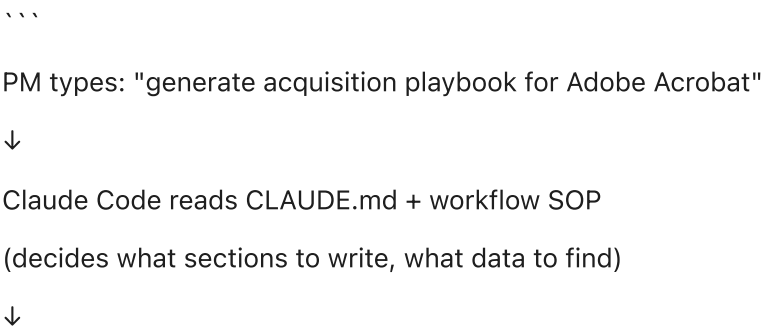
2. CONTACTS

Name	Role	Responsibility
Bharat Namatherdhala	Adobe PM (Owner)	Requirements, report consumer, stakeholder demos
Claude Sonnet 4.6	AI Agent (Builder)	Code, automation, ongoing maintenance

3. HOW A PDF REPORT IS GENERATED — FULL PIPELINE

There are two modes: **Automated** (runs in the cloud via Trigger.dev) and **Manual** (runs locally via Claude Code). Both produce the same output.

Mode A — Manual Pipeline (Claude Code, used for all current playbooks)



API Call 1: WebSearch (Brave Search API)

→ Query: "Adobe Acrobat pricing 2026"

→ Query: "Foxit PDF Editor pricing 2026"

→ Query: "UPDF vs Adobe Acrobat comparison"

→ Query: "Adobe Acrobat churn complaints Reddit"

→ Returns: up to 10 live web results per query

↓

Claude Sonnet 4.6 synthesizes results into 13-section markdown

(pricing tables, battlecards, PLG motions, metrics dashboard)

↓

Claude writes: docs/Acrobat-Acquisition-Playbook.md

↓

Script call: npx tsx scripts/md-to-pdf.ts docs/Acrobat-Acquisition-Playbook.md

↓

md-to-pdf.ts (Puppeteer):

→ Reads markdown file

→ Converts markdown → HTML with inline CSS (system fonts, Adobe red header)

→ Puppeteer launches headless Chrome

→ page.setContent(html, { waitUntil: "networkidle0" })

→ page.pdf({ format: "A4", printBackground: true })

→ Saves PDF → ~/Desktop/Adobe-Acrobat-Acquisition-Playbook-2026-04.pdf

↓

PM opens PDF on Desktop

...

APIs called per report (manual mode):

API	Purpose	Cost	Auth
Brave Search API	Live web research (pricing, competitor sites, Reddit threads)	Free tier (2,000 queries/month)	<code>BRAVE_API_KEY</code> in <code>.env</code>
Anthropic Claude Sonnet 4.6	Synthesis, writing, structuring the full report	~\$0.08–0.15/report	<code>ANTHROPIC_API_KEY</code> in <code>.env</code>
Puppeteer (headless Chrome)	HTML → PDF rendering	Free (local)	None

Mode B — Automated Pipeline (Trigger.dev, used for scheduled Firefly reports)

...

Trigger.dev CRON fires: 1st of every month, 9am PT

↓

orchestrator.ts (main task) starts

↓

└─ batchTriggerAndWait([

| reddit.ts → Reddit public JSON API (no key needed)

| searches: "{competitor} credits pricing"

| returns: top 3 posts per competitor

|

| youtube.ts → YouTube Data API v3

| searches: "{competitor} AI image review 2026"

| returns: 3 recent video titles + descriptions

|

| web-search.ts → Brave Search API

| searches: "{competitor} pricing site:{competitor}.com"

| returns: 5 results per competitor

|])

└─ (all 7 competitors scraped in parallel, ~15 seconds)

↓

analyzer.ts → Anthropic Claude Haiku

→ Input: all scraped data, hard-capped at 6,000 tokens

→ Prompt: structured JSON schema for competitive analysis

→ Output: JSON with executive summary, pricing matrix, recommendations

↓

report-template.ts → builds HTML from JSON

→ Adobe brand colors, dark canvas theme, structured sections

↓

Trigger.dev stores run output (accessible via Management API)

↓

(Manual step) PM runs: npm run save-report

→ save-report.ts fetches latest run output via Trigger.dev API

→ Puppeteer renders HTML → PDF

→ Saves to ~/Desktop/

...

APIs called per report (automated mode):

API	Purpose	Cost	Auth
Reddit public JSON API	Community sentiment on competitor credit packs	Free (no key)	None
YouTube Data API v3	Video reviews and tutorials per competitor	Free (10K units/day)	<code>YOUTUBE_API_KEY</code>
Brave Search API	Live pricing and news per competitor	Free (2K queries/month)	<code>BRAVE_API_KEY</code>
Anthropic Claude Haiku	Analysis synthesis — cheapest capable model	~\$0.003/report	<code>ANTHROPIC_API_KEY</code>
Trigger.dev Management API	Fetch run output for PDF generation	Included in plan	<code>TRIGGER_API_KEY</code>
Puppeteer (headless Chrome)	HTML → PDF	Free (local)	None

Total automated cost per report: ~\$0.003 — less than one-third of one cent.

PDF Generation Deep Dive (scripts/md-to-pdf.ts)

The PDF script does the following in order:

...

- 1. Reads markdown file from disk (readFileSync)
- 2. Parses markdown → HTML:
 - Headings (#, ##, ###) →

,

- **Bold/italic** → ,
- **Tables** (`|col|col|`) →
- **Lists** (`-`, `1.`) →
- ,
- **Blockquotes** (`>`) →

(used for TL;DR callout box)

1. Wraps in full HTML document with inline CSS:

- Font: system fonts only (-apple-system, BlinkMacSystemFont, Segoe UI, Arial)

→ No Google Fonts = no external requests = PDF renders reliably

- Theme: white background, #1E1E1E text

- Adobe header: red (#EB1000) background, "Adobe" logo, report title

- Tables: dark header row, alternating row shading

- Print CSS: @page { size: A4; margin: 0 }

1. Puppeteer launches headless Chrome

2. page.setContent(html, { waitUntil: "networkidle0" })

→ waitUntil: "networkidle0" ensures all rendering is complete before capture

1. page.pdf({ format: "A4", printBackground: true, margin: 0 })

2. Saves to ~/Desktop/Adobe-{filename}-{YYYY-MM}.pdf

...

Why system fonts? Earlier versions loaded Google Fonts (`Source Sans 3`) via a `<link>` tag. Headless Chrome cannot reliably load external fonts before PDF capture, causing blank/empty PDFs. System fonts render instantly with no network dependency.

4. MCP INTEGRATIONS

MCP (Model Context Protocol) allows Claude to call external tools and services as part of its reasoning loop — without writing separate glue code. The following MCPs are used or planned:

Currently Active

MCP	What it does in this system	Where used
WebSearch (Brave)	Claude calls live web search mid-conversation to verify current pricing, find competitor pages, and pull real data before writing reports	All manual playbook generation
Filesystem	Claude reads/writes files directly — markdown reports, markdown sources, config files	Writing <code>.md</code> files, reading workflow SOPs
Bash/Shell	Claude runs <code>npx tsx scripts/md-to-pdf.ts</code> directly from the conversation to trigger PDF generation	PDF generation step

Phase 2 — Planned MCP Integrations

MCP	Purpose	Required for
Slack MCP	Post formatted report summaries to #competitive-intel channel with PDF attachment	Phase 2 Slack delivery
Google Drive MCP	Save PDFs to a shared Drive folder; auto-generate shareable link	Phase 2 distribution
GitHub MCP	Commit new report files, create PRs for report updates without leaving the Claude session	Phase 2 CI/CD
Email (Resend MCP)	Send report to distribution list on the 2nd of each month	Phase 2 distribution

Phase 3 — Advanced MCP Integrations

MCP	Purpose
Notion MCP	Sync report sections directly into a Notion competitive intel database
Linear MCP	Auto-create PM tickets from PLG recommendations in each report
Airtable MCP	Log per-competitor metrics to a shared Airtable base for trend tracking
Confluence MCP	Publish reports to Adobe's internal wiki for broader team access

5. CURRENT DELIVERABLES (PHASE 1 COMPLETE)

All three PDF playbooks are live in the repo and downloadable at the Vercel site.

Report	File	Pages	Key Sections
Acrobat Acquisition Playbook	Adobe-Acrobat-Acquisition-Playbook-2026-04.pdf	~8	Pricing · 3 Battlecards · PLG Now/3M/6M · 8 Metrics
Express Acquisition Playbook	Adobe-Express-Acquisition-Playbook-2026-04.pdf	~8	Pricing · vs Canva/M365/Picsart · PLG loops · 8 Metrics
Remove Background Playbook	Adobe-Remove-Background-Playbook-2026-04.pdf	~8	18M keyword · SEO sprint · Shopify app · 8 Metrics
Firefly Automated Report	Generated monthly via Trigger.dev	~4	Competitor pricing matrix · Reddit sentiment · PM recommendations

Markdown sources (in docs/):

- docs/Acrobat-Acquisition-Playbook.md
- docs/Express-Acquisition-Playbook.md
- docs/Remove-Background-Playbook.md

Demo resources (in docs/):

- docs/DEMO-GUIDE.md — 10-minute demo script + Q&A prep + "how to use in your Claude account"

6. PHASE ROADMAP

Phase 1 — Local Generation (COMPLETE)

Goal: Replace manual competitive research with AI-generated PDFs on demand.

What was built:

- **Reddit + Brave + YouTube scraper for Firefly competitors (automated via Trigger.dev)**
- **Claude-generated acquisition playbooks for Acrobat, Express, and Remove Background keyword**
- `md-to-pdf.ts` — **markdown → Adobe-branded PDF in one command**
- **CLAUDE.md + workflow SOPs — plain-English instructions that make Claude a consistent agent**
- **GitHub repo + Vercel deployment — PDFs downloadable at public URL**
- **DEMO-GUIDE.md — 10-minute demo for org presentation**

How to run:

```
``bash
npm run test-run
npx tsx scripts/md-to-pdf.ts docs/[filename].md
````
```

**Cost:** ~\$0.003–0.15/report. Under \$2/year at monthly cadence.

---

### Phase 2 — Slack Integration (NEXT)

**Goal:** Reports land in the right Slack channels automatically, with no manual steps after generation.

**What to build:**

#### 2.1 — Automated monthly Slack delivery

**Every 1st of the month, Trigger.dev fires the Firefly competitor pipeline and posts the executive summary directly to `#competitive-intel`:**

...

**Competitive Intel — May 2026** ⚡

---

- **Midjourney raised Standard plan from \$30 → \$35/mo (15% increase)**
- **DALL-E added real-time generation; early reviews cite 3x speed vs Firefly**
- **Canva AI added bulk generation to Pro; community reaction mixed on quality**
- **Ideogram 3.0 launched with photorealism claims — Reddit positive, skeptical on commercial use**

- **Adobe Firefly: commercial safety remains unchallenged; CC integration praised in r/photography**

 **Full report: [Download PDF](#)**

 **Source: [github.com/bnamatherdhala7/Competitive-research-Report-using-Claude](https://github.com/bnamatherdhala7/Competitive-research-Report-using-Claude)**

...

**API calls added in Phase 2:**

| New API                                         | Purpose                                            | Auth                                              |
|-------------------------------------------------|----------------------------------------------------|---------------------------------------------------|
| Slack Web API ( <code>chat.postMessage</code> ) | Post summary to channel                            | <code>SLACK_BOT_TOKEN</code> in <code>.env</code> |
| Slack Files API ( <code>files.upload</code> )   | Upload PDF as Slack file attachment                | <code>SLACK_BOT_TOKEN</code> in <code>.env</code> |
| Google Drive API (optional)                     | Save PDF to shared folder, generate shareable link | <code>GOOGLE_SERVICE_ACCOUNT_JSON</code>          |
| Resend API (optional)                           | Send PDF by email to distribution list             | <code>RESEND_API_KEY</code>                       |

**Trigger.dev task changes:**

```
```typescript
```

```
// src/trigger/competitor-analysis/orchestrator.ts — Phase 2 additions
```

```
// After PDF is generated:
```

```
await slack.postMessage({
  channel: "#competitive-intel",
  text: buildSlackSummary(report), // 5-bullet executive summary
  attachments: [{ title: "Full PDF Report", ...pdfAttachment }]
});

// Optional: save to Drive
const driveLink = await uploadToGoogleDrive(pdfBuffer, reportTitle);
await slack.postMessage({ channel: "#competitive-intel", text:  PDF saved: ${driveLink} });
...
```

2.2 — On-demand Slack bot command

A Slack slash command `/competitive-intel [product]` triggers a new report:

...

User types: `/competitive-intel Adobe Express`

↓

Slack sends webhook to Trigger.dev

↓

`orchestrator.ts` runs Express pipeline

↓

Slack bot replies in thread: "Generating... (est. 45 seconds)"

↓

Report ready → bot posts PDF + 5-bullet summary in thread

...

Implementation:

- Slack App with slash command `/competitive-intel`
- Slash command webhook → Trigger.dev HTTP endpoint
- Trigger.dev task runs pipeline → posts result back to Slack thread

2.3 — Targeted channel routing

Report type	Slack channel
Acrobat updates	<code>#acrobat-pm</code>
Express updates	<code>#express-growth</code>
Firefly updates	<code>#firefly-competitive-intel</code>
All reports (digest)	<code>#pm-competitive-intel</code>

Phase 2 effort estimate: 3–5 days of Claude Code work.

Phase 3 — Intelligence Platform

Goal: Trend tracking, cross-team access, and a self-service dashboard — making this the source of truth for Adobe competitive intelligence across product teams.

What to build:

3.1 — Trend detection between reports

Each month's report is compared to the previous month. Claude highlights what changed:

...

CHANGES DETECTED — May vs April 2026

- 🔴 Midjourney: price +17% (Standard \$30→\$35) — flagged for Firefly pricing review
- 🟡 DALL-E: launched real-time generation — competitive threat in speed positioning
- 🟢 Adobe Firefly: community sentiment improved +12% — commercial safety narrative landing

...

Implementation:

- Store each report's JSON output in a database (PlanetScale or Supabase free tier)
- On each new run, Claude Haiku diffs current vs previous JSON
- Delta summary prepended to report and Slack post

3.2 — Web dashboard

A simple read-only dashboard at `competitive-research-report-using-c.vercel.app/dashboard` showing:

- Report archive (all past reports, downloadable)
- Trend charts per competitor (pricing over time, sentiment score over time)
- Key metrics table updated monthly

Tech: Next.js (or plain HTML) + Vercel + Supabase for JSON storage.

3.3 — Notion / Confluence integration

Each report's key sections (TL;DR, pricing table, battlecards, PLG recommendations) are synced to a Notion database or Confluence page — so teams who live in those tools get the intelligence without needing to find the PDF.

3.4 — Linear ticket auto-creation

PLG recommendations from each report automatically create Linear issues:

...

Title: [PLG] Cancel-intervention modal — Acrobat retention

Body: From April 2026 competitive report — cancel intervention modal can lift retention from 10% → 30%. Counter: Foxit/UPDF no-ETF pitch.

Metric: cancellation-to-retention rate.

Labels: competitive-intel, plg, acrobat

...

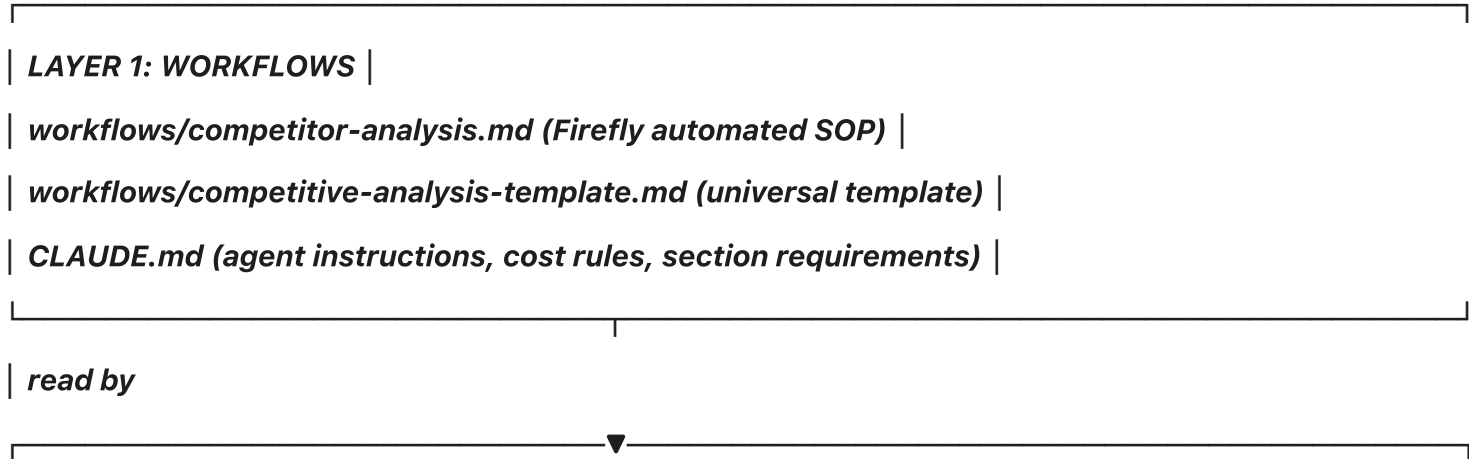
3.5 — Expanded product coverage

Extend the automated pipeline beyond Firefly to cover Acrobat, Express, and Remove Background keyword monthly — using the same Trigger.dev + Brave + Claude architecture, with each product's playbook template as the synthesis prompt.

Phase 3 effort estimate: 2–3 weeks of Claude Code work.

7. FULL SYSTEM ARCHITECTURE

...



| LAYER 2: AGENT |

Claude Sonnet 4.6 (Claude Code, local)

→ Reads workflow SOP, decides what to research and write

→ *Calls WebSearch MCP for live pricing and competitor data*

```
→ Writes markdown report to docs/
```

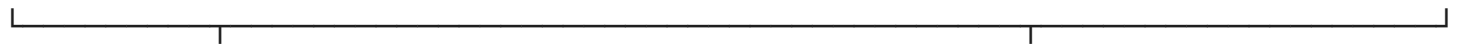
→ Calls Bash MCP to run md-to-pdf.ts

11

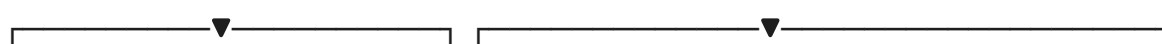
Claude Haiku (Trigger.dev, cloud)

→ *Receives pre-scraped data from orchestrator*

→ Returns structured JSON analysis



calls (Manual) | calls (Automated)



LAYER 3: TOOLS | LAYER 3: TOOLS

(Local Scripts) | | (Trigger.dev Tasks) |

|||

```
| scripts/ | | src/trigger/ |
```

md-to-pdf.ts | orchestrator.ts

<i>test-run.ts</i>	<i>reddit.ts</i>
--------------------	------------------

| [save-report.ts](#) | | [youtube.ts](#) |

```
| | | web-search.ts |
```

APIs: | *analyzer.ts* |

Brave Search | **report-template.ts**

| **Puppeteer** | | |

Anthropic API | **APIs:**

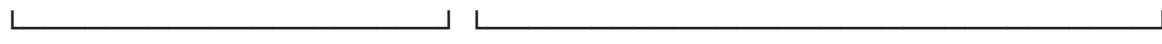
| | | *Reddit public JSON* |

YouTube Data API v3

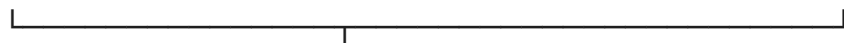
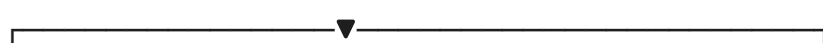
Brave Search API

Anthropic Haiku API

Trigger.dev Management API



11

| **outputs**

| **DELIVERABLES** |

| *~/Desktop/Adobe-*.pdf (local)* |

| *docs/*.pdf (repo + Vercel)* |

| *docs/*.md (markdown sources)* |

| *Trigger.dev run logs (cloud)* |

Phase 2 adds ↓

| *Slack: #competitive-intel channel* |

| *Google Drive: shared PDF folder* |

| *Email: distribution list (Resend)* |

Phase 3 adds ↓

| *Dashboard: vercel.app/dashboard* |

| *Notion / Confluence sync* |

| *Linear tickets from PLG recs* |

| *Supabase: historical JSON storage* |

...

8. ENVIRONMENT VARIABLES

All secrets live in `.env` only — never in code, never in comments, never in logs.

Variable	Used for	Where to get it
<code>ANTHROPIC_API_KEY</code>	Claude Haiku (analysis) + Claude Sonnet (manual)	console.anthropic.com
<code>BRAVE_API_KEY</code>	Brave Search (web research)	api.search.brave.com
<code>YOUTUBE_API_KEY</code>	YouTube Data API v3 (video search)	console.cloud.google.com
<code>TRIGGER_API_KEY</code>	Trigger.dev Management API	cloud.trigger.dev
<code>TRIGGER_PROJECT_ID</code>	Trigger.dev project identifier	cloud.trigger.dev
<code>SLACK_BOT_TOKEN</code>	(Phase 2) Slack Web API	api.slack.com/apps
<code>SLACK_CHANNEL_ID</code>	(Phase 2) Target Slack channel	Slack app settings
<code>RESEND_API_KEY</code>	(Phase 2) Email delivery	resend.com
<code>GOOGLE_SERVICE_ACCOUNT_JSON</code>	(Phase 2) Google Drive upload	Google Cloud Console

9. COST SUMMARY

Mode	Per report	Monthly (1 report)	Annual (12 reports)
Manual (Claude Sonnet)	~\$0.10–0.15	~\$0.12	~\$1.44
Automated (Claude Haiku)	~\$0.003	~\$0.003	~\$0.036
Phase 2 (+ Slack, Drive)	+\$0	Slack free tier	Included
Phase 3 (+ Supabase, Linear)	+\$0	Supabase free tier	Included

Hard cap: Never exceed \$0.50/report. Stop if Anthropic API costs spike.

10. HOW TO USE THIS SYSTEM IN YOUR CLAUDE ACCOUNT

Anyone on the team with a Claude account can generate a new competitive playbook:

Option 1 — Claude.ai (no setup, 2 minutes)

Paste this into Claude at claude.ai:

"You are a competitive intelligence analyst for Adobe. Generate a focused acquisition playbook for [product name]. Include: TL;DR (2–3 sentences), pricing comparison table vs top 5–6 competitors with real prices, competitor website messaging, top 5 churn triggers with rank, win signals, 3 battlecards (win move + lose scenario per competitor), PLG acquisition strategy with Now/3M/6M moves (each must name the competitor countered, what to build, and what metric to track), and a PLG metrics dashboard table with current estimates and 6-month targets. Use web search to verify current pricing."

Option 2 — Claude Code CLI (generates PDF automatically)

```
```bash
npm install -g @anthropic-ai/claude-code
cd path/to/Competative-Intelligence
claude
```
```

Option 3 — Full automated pipeline (requires API keys)

```
```bash
git clone https://github.com/bnamatherdhala7/Competitive-research-Report-using-Claude
cd Competitive-research-Report-using-Claude
npm install
echo "ANTHROPIC_API_KEY=your_key" >> .env
```

```
echo "BRAVE_API_KEY=your_key" >> .env
```

```
npm run test-run # scrape + analyze
```

```
npm run save-report # generate PDF → Desktop
```

```
...
```